Mutex and Locking Mechanisms - Synchronization Primitives

Theory, Edge Cases & Code Examples

- ✓ Mutex Overview
- ✓ The Need for Synchronization
- ✓ The Modern Way
- ✓ Mutex Types
- ✓ Mutex Design in Real-Time Systems

→ Swipe to learn →

Mutex and Locking Mechanisms - Synchronization Primitives

Theory

📌 1. Mutex Overview

Definition:

- **Mutex** = Mutual Exclusion
- Fundamental synchronization primitive
- Ensures only ONE thread accesses shared resource at a time
- Protects critical sections from concurrent access

The Problem: Race Conditions

```
// ❌ WITHOUT MUTEX - Race condition
int counter = 0;

void increment() {
    ++counter;  // NOT atomic! Actually 3 operations:
                // 1. Load counter from memory
                // 2. Increment in register
                // 3. Store back to memory
}

// Multiple threads → lost updates
```

Core Mechanism:

Operation	Effect	Thread Behavior
`lock()`	Acquire exclusive access	Blocks if already locked
`unlock()`	Release exclusive access	Wakes waiting threads
Critical section	Code between lock/unlock	Only one thread executes

---

Mutex and Locking Mechanisms - Synchronization Primitives

Theory

📌 2. The Need for Synchronization

What is a Data Race?

- Multiple threads access shared data concurrently
- At least one is writing
- No synchronization mechanism
- **Result:** Undefined behavior, data corruption

Example: Counter Increment

```
Thread 1                Thread 2
--------                --------
Read counter (0)        Read counter (0)
Increment (1)           Increment (1)
Write counter (1)       Write counter (1)

Expected: 2
Actual: 1 (lost update!)
```

Common Data Race Scenarios:

Scenario	Problem	Solution
Shared counter	Lost updates	Mutex or atomic
Shared container	Corruption, crashes	Mutex protection
Config reload	Partial reads	Readers-writer lock
Bank transfer	Inconsistent state	Multi-mutex locking

Mutex and Locking Mechanisms - Synchronization Primitives

Theory

📌 3. RAII Locking - The Modern Way (1/2)

std::lock_guard - Simple RAII

```
std::mutex mtx;

void safe_increment() {
    std::lock_guard<std::mutex> lock(mtx);  // ✅ Locks on construction
    ++counter;
    // Exception-safe: always unlocks on destruction
}  // ← Automatic unlock here
```

Characteristics:

Feature	lock_guard	Manual lock/unlock
Exception Safety	✅ Automatic	❌ Must use try-catch
Early Return Safety	✅ Automatic	❌ Must unlock before return
Overhead	Zero (compiler optimized)	Same
Flexibility	Limited	Full control

std::unique_lock - Flexible RAII

```
void flexible_locking() {
    std::unique_lock<std::mutex> lock(mtx);  // Locked

    critical_work();

    lock.unlock();  // ✅ Manual unlock
    non_critical_work();
    lock.lock();    // ✅ Re-lock

    more_critical_work();
}  // Automatic unlock
```

Mutex and Locking Mechanisms - Synchronization Primitives

Theory

📌 3. RAII Locking - The Modern Way (2/2)

When to Use Each:

Lock Type	Use When...
`lock_guard`	Simple scope-based locking
`unique_lock`	Need manual lock/unlock control
`unique_lock`	Using with condition variables
`unique_lock`	Need deferred or timed locking

Mutex and Locking Mechanisms - Synchronization Primitives

Theory

📌 4. Mutex Types (1/3)

Available Mutex Types:

Mutex Type	Recursive	Timeout	Shared Lock	Use Case
`std::mutex`	❌	❌	❌	Basic mutual exclusion
`std::recursive_mutex`	✅	❌	❌	Nested locking (same thread)
`std::timed_mutex`	❌	✅	❌	Timeout-based locking
`std::shared_mutex` (C++17)	❌	❌	✅	Multiple readers, one writer

std::recursive_mutex Example:

```
std::recursive_mutex rmtx;

void outer() {
    std::lock_guard<std::recursive_mutex> lock(rmtx);
    inner();  // ✅ OK - can relock
}

void inner() {
    std::lock_guard<std::recursive_mutex> lock(rmtx);  // Same thread
    // Works because mutex is recursive
}
```

Mutex and Locking Mechanisms - Synchronization Primitives

Theory

📌 4. Mutex Types (2/3)

std::timed_mutex Example:

```
std::timed_mutex tmtx;

if (tmtx.try_lock_for(std::chrono::milliseconds(100))) {
    // ✅ Got lock within 100ms
    critical_section();
    tmtx.unlock();
} else {
    // ❌ Timeout - handle gracefully
    log_contention();
}
```

std::shared_mutex Example (Readers-Writer Lock):

```
std::shared_mutex smtx;
std::unordered_map<int, std::string> cache;

void read(int key) {
    std::shared_lock<std::shared_mutex> lock(smtx);  // ✅ Multiple readers OK
    auto value = cache[key];
}

void write(int key, std::string value) {
    std::unique_lock<std::shared_mutex> lock(smtx);  // Exclusive writer
    cache[key] = value;
}
```

Mutex and Locking Mechanisms - Synchronization Primitives

Theory

📌 4. Mutex Types (3/3)

Performance Trade-offs:

Mutex Type	Overhead	Notes
`std::mutex`	Lowest	Best for simple cases
`std::recursive_mutex`	~1.5-2x	Tracks ownership + lock count
`std::timed_mutex`	~1.5-2x	Timeout logic overhead
`std::shared_mutex`	~2-3x reads, ~3-4x writes	Bookkeeping for multiple readers

---

Mutex and Locking Mechanisms - Synchronization Primitives

Theory

📌 5. Mutex Design in Real-Time Systems (1/2)

Autonomous Vehicle Example:

```
Sensor Threads:          Shared World Model          Planning Thread:
- Camera (30 Hz)  ┐                                 ┌─ Path Planning (10 Hz)
- LiDAR (10 Hz)   ├──> [Protected by Mutex] <───────┤
- Radar (20 Hz)   ┘                                 └─ Uses latest data
```

Lock Granularity Trade-off:

Approach	Pros	Cons
Coarse-grained (One mutex for all)	Simple, no deadlock	High contention, poor scalability
Fine-grained (Mutex per region)	Low contention, good scalability	Complex, deadlock risk

Coarse-Grained Example:

```
// ❌ One mutex protects entire world model
std::mutex world_mutex;
WorldModel world;

void update_camera_data() {
    std::lock_guard<std::mutex> lock(world_mutex);  // Blocks everything
    world.update_vision();
}

void update_lidar_data() {
    std::lock_guard<std::mutex> lock(world_mutex);  // Also blocks everything
    world.update_lidar();
}
// Problem: Updates serialize even though they touch different data
```

Mutex and Locking Mechanisms - Synchronization Primitives

Theory

📌 5. Mutex Design in Real-Time Systems (2/2)

Fine-Grained Example:

```
// ✅ Separate mutexes for different data
struct WorldModel {
    std::mutex vision_mutex;
    std::mutex lidar_mutex;
    VisionData vision;
    LiDARData lidar;
};

void update_camera_data() {
    std::lock_guard<std::mutex> lock(world.vision_mutex);  // Only locks vision
    world.vision.update();
}

void update_lidar_data() {
    std::lock_guard<std::mutex> lock(world.lidar_mutex);  // Only locks lidar
    world.lidar.update();
}
// Benefit: Parallel updates possible
```

Critical Requirements:

- **Correctness:** No data races, consistent state
- **Performance:** Minimize lock hold time
- **Latency:** Bounded wait times for real-time threads
- **Deadlock Freedom:** Careful lock ordering or std::lock()

Mutex and Locking Mechanisms - Synchronization Primitives

Theory

📌 6. Key Concepts Summary

RAII Lock Wrappers:

Wrapper	Lock on Construction	Manual Lock/Unlock	Movable	Use Case
`lock_guard`	✅	❌	❌	Simple scope lock
`unique_lock`	✅ (unless defer_lock)	✅	✅	Flexible locking
`scoped_lock` (C++17)	✅	❌	❌	Multi-mutex lock

Locking Strategies:

```
// Strategy 1: Simple scope lock
{
    std::lock_guard<std::mutex> lock(mtx);
    // Critical section
}  // Auto unlock

// Strategy 2: Deferred lock
std::unique_lock<std::mutex> lock(mtx, std::defer_lock);
// ... preparation ...
lock.lock();  // Lock when ready

// Strategy 3: Multi-mutex (deadlock-free)
std::lock(mtx1, mtx2);  // Atomic acquisition
std::lock_guard<std::mutex> lock1(mtx1, std::adopt_lock);
std::lock_guard<std::mutex> lock2(mtx2, std::adopt_lock);
```

Mutex and Locking Mechanisms - Synchronization Primitives

Edge Cases

📌 1. Exception Safety Without RAII

```
std::mutex mtx;
int shared_counter = 0;

void unsafe_increment() {
    mtx.lock();
    ++shared_counter;

    if (shared_counter == 100) {
        throw std::runtime_error("Error!");  // ❌ Lock never released!
    }

    mtx.unlock();  // Never reached if exception thrown
}
```

If an exception is thrown between `lock()` and `unlock()`, the mutex remains locked forever, causing deadlock when any other thread attempts to acquire it. This is why manual locking is dangerous.

Solution with RAII:

```
void safe_increment() {
    std::lock_guard<std::mutex> lock(mtx);  // ✅ Auto-unlocks on exception
    ++shared_counter;

    if (shared_counter == 100) {
        throw std::runtime_error("Error!");  // Lock released by destructor
    }
}
```

The `lock_guard` destructor is called during stack unwinding, ensuring the mutex is always released.

Mutex and Locking Mechanisms - Synchronization Primitives

Edge Cases

📌 2. Self-Deadlock with Non-Recursive Mutex

```
std::mutex mtx;

void inner_function() {
    std::lock_guard<std::mutex> lock(mtx);  // ❌ Tries to lock already-held mutex
    // ... code ...
}

void outer_function() {
    std::lock_guard<std::mutex> lock(mtx);
    inner_function();  // Deadlock! Same thread tries to reacquire same mutex
}
```

Standard `std::mutex` does not support recursive locking. When the same thread attempts to lock it twice, it deadlocks waiting for itself to release the lock.

Solutions:

1. Use `std::recursive_mutex` (allows same thread to lock multiple times)
2. Refactor to avoid nested locking (better design)

```
std::recursive_mutex rmtx;

void inner_function() {
    std::lock_guard<std::recursive_mutex> lock(rmtx);  // ✅ OK with recursive_mutex
    // ... code ...
}

void outer_function() {
    std::lock_guard<std::recursive_mutex> lock(rmtx);
    inner_function();  // No deadlock - recursive locking allowed
}
```

Note: Recursive mutexes have higher overhead and can hide design issues. Prefer refactoring over recursive mutexes.

Mutex and Locking Mechanisms - Synchronization Primitives

Edge Cases

📌 3. Lock Ordering Deadlock (1/2)

```
std::mutex mtx1, mtx2;

void thread_A() {
    std::lock_guard<std::mutex> lock1(mtx1);  // Lock mtx1 first
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
    std::lock_guard<std::mutex> lock2(mtx2);  // Then lock mtx2
    // Critical section
}

void thread_B() {
    std::lock_guard<std::mutex> lock2(mtx2);  // ❌ Lock mtx2 first
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
    std::lock_guard<std::mutex> lock1(mtx1);  // Then lock mtx1 → DEADLOCK
    // Critical section
}
```

Classic deadlock scenario: Thread A holds mtx1 and waits for mtx2; Thread B holds mtx2 and waits for mtx1. Neither can proceed.

Solution 1: Consistent Lock Ordering

```
// Both threads acquire locks in same order
void thread_A() {
    std::lock_guard<std::mutex> lock1(mtx1);  // ✅ Same order
    std::lock_guard<std::mutex> lock2(mtx2);
}

void thread_B() {
    std::lock_guard<std::mutex> lock1(mtx1);  // ✅ Same order
    std::lock_guard<std::mutex> lock2(mtx2);
}
```

Mutex and Locking Mechanisms - Synchronization Primitives

Edge Cases

📌 3. Lock Ordering Deadlock (2/2)

Solution 2: std::lock for Atomic Multi-Lock

```
void thread_A() {
    std::lock(mtx1, mtx2);  // ✅ Atomically locks both, deadlock-free
    std::lock_guard<std::mutex> lock1(mtx1, std::adopt_lock);
    std::lock_guard<std::mutex> lock2(mtx2, std::adopt_lock);
}

void thread_B() {
    std::lock(mtx1, mtx2);  // ✅ Same - deadlock-free
    std::lock_guard<std::mutex> lock1(mtx1, std::adopt_lock);
    std::lock_guard<std::mutex> lock2(mtx2, std::adopt_lock);
}
```

Mutex and Locking Mechanisms - Synchronization Primitives

Edge Cases

📌 4. Forgetting std::adopt_lock After std::lock

```
std::mutex mtx1, mtx2;

void incorrect_usage() {
    std::lock(mtx1, mtx2);  // ✅ Both mutexes now locked

    std::lock_guard<std::mutex> lock1(mtx1);  // ❌ Tries to lock again → deadlock!
    std::lock_guard<std::mutex> lock2(mtx2);
}

void correct_usage() {
    std::lock(mtx1, mtx2);

    std::lock_guard<std::mutex> lock1(mtx1, std::adopt_lock);  // ✅ Don't re-lock
    std::lock_guard<std::mutex> lock2(mtx2, std::adopt_lock);
}
```

`std::adopt_lock` tells the guard "the mutex is already locked, just manage its lifetime, don't lock it again."

Mutex and Locking Mechanisms - Synchronization Primitives

Edge Cases

📌 5. Mutex Granularity Trade-off

```
// ❌ Too coarse-grained - single mutex for entire data structure
class CoarseGrainedMap {
    std::mutex mtx;
    std::unordered_map<int, std::string> data;
public:
    void insert(int key, std::string value) {
        std::lock_guard<std::mutex> lock(mtx);  // Blocks ALL operations
        data[key] = value;
    }

    std::string lookup(int key) {
        std::lock_guard<std::mutex> lock(mtx);  // Even lookups block each other
        return data[key];
    }
};

// ✅ Fine-grained with shared_mutex - multiple readers, single writer
class FineGrainedMap {
    std::shared_mutex mtx;
    std::unordered_map<int, std::string> data;
public:
    void insert(int key, std::string value) {
        std::unique_lock<std::shared_mutex> lock(mtx);  // Exclusive lock
        data[key] = value;
    }

    std::string lookup(int key) {
        std::shared_lock<std::shared_mutex> lock(mtx);  // ✅ Shared lock - multiple readers OK
        return data[key];
    }
};
```

Coarse-grained locking is simple but creates contention. Fine-grained locking (readers-writer locks, per-bucket locks) improves parallelism but increases complexity.

Mutex and Locking Mechanisms - Synchronization Primitives

Code Examples

📌 1. Race Condition Without Synchronization

Without synchronization, the counter value is unpredictable, typically much less than 1,000,000 due to lost updates from concurrent increments.

```
#include <iostream>
#include <thread>
#include <vector>

int counter = 0;  // Shared mutable state

void increment_unsafe() {
    for (int i = 0; i < 100000; ++i) {
        ++counter;  // ❌ Race condition - not atomic
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 10; ++i) {
        threads.emplace_back(increment_unsafe);
    }

    for (auto& t : threads) {
        t.join();
    }

    std::cout << "Counter: " << counter << "\n";  // Expected: 1000000, Actual: varies!
}
```

Mutex and Locking Mechanisms - Synchronization Primitives

Code Examples

📌 2. Fixing Race Condition with std::mutex

Manual lock/unlock works but is error-prone. Prefer RAII wrappers.

```
#include <iostream>
#include <thread>
#include <vector>
#include <mutex>

std::mutex mtx;
int counter = 0;

void increment_safe() {
    for (int i = 0; i < 100000; ++i) {
        mtx.lock();       // Acquire exclusive access
        ++counter;        // ✅ Safe - only one thread at a time
        mtx.unlock();     // Release lock
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 10; ++i) {
        threads.emplace_back(increment_safe);
    }

    for (auto& t : threads) {
        t.join();
    }

    std::cout << "Counter: " << counter << "\n";  // Always 1000000
}
```

Mutex and Locking Mechanisms - Synchronization Primitives

Code Examples

📌 3. Exception-Safe Locking with std::lock_guard

The `lock_guard` destructor releases the mutex even when exceptions are thrown, preventing deadlocks.

```
#include <iostream>
#include <thread>
#include <mutex>
#include <stdexcept>

std::mutex mtx;
int counter = 0;

void increment_with_exception() {
    for (int i = 0; i < 100; ++i) {
        std::lock_guard<std::mutex> lock(mtx);  // ✅ RAII - auto-unlocks
        ++counter;

        if (counter == 50) {
            throw std::runtime_error("Simulated error");  // Lock released automatically
        }
    }
}

int main() {
    try {
        std::thread t1(increment_with_exception);
        std::thread t2(increment_with_exception);

        t1.join();
        t2.join();
    } catch (const std::exception& e) {
        std::cout << "Caught: " << e.what() << "\n";
    }

    std::cout << "Counter: " << counter << "\n";  // Lock was released despite exception
}
```

Mutex and Locking Mechanisms - Synchronization Primitives

Code Examples

📌 4. Protecting Shared Container

All operations on the shared container are protected by the same mutex, ensuring thread safety.

```
#include <iostream>
#include <thread>
#include <vector>
#include <mutex>

std::mutex mtx;
std::vector<int> shared_data;

void add_data(int value) {
    std::lock_guard<std::mutex> lock(mtx);
    shared_data.push_back(value);  // ✅ Thread-safe
}

void print_data() {
    std::lock_guard<std::mutex> lock(mtx);
    for (int val : shared_data) {
        std::cout << val << " ";
    }
    std::cout << "\n";
}

int main() {
    std::thread t1(add_data, 1);
    std::thread t2(add_data, 2);
    std::thread t3(add_data, 3);

    t1.join();
    t2.join();
    t3.join();

    print_data();  // Output: 1 2 3 (order may vary)
}
```

Mutex and Locking Mechanisms - Synchronization Primitives

Code Examples

📌 5. std::unique_lock for Flexible Locking

`std::unique_lock` allows manual lock/unlock, useful for minimizing lock hold time by releasing during non-critical work.

```
#include <iostream>
#include <thread>
#include <mutex>

std::mutex mtx;

void flexible_locking() {
    std::unique_lock<std::mutex> lock(mtx);  // Locked

    // Critical section 1
    std::cout << "First critical section\n";

    lock.unlock();  // ✅ Explicitly unlock

    // Non-critical work
    std::this_thread::sleep_for(std::chrono::milliseconds(10));

    lock.lock();  // ✅ Re-lock

    // Critical section 2
    std::cout << "Second critical section\n";

    // Automatically unlocks when lock goes out of scope
}

int main() {
    std::thread t1(flexible_locking);
    std::thread t2(flexible_locking);

    t1.join();
    t2.join();
}
```

Mutex and Locking Mechanisms - Synchronization Primitives

Code Examples

📌 6. Deferred Locking

`std::defer_lock` creates an unlocked `unique_lock`, allowing you to control when locking occurs.

```
#include <iostream>
#include <thread>
#include <mutex>

std::mutex mtx;

void deferred_lock_example() {
    std::unique_lock<std::mutex> lock(mtx, std::defer_lock);  // ✅ Don't lock yet

    // Do some work without holding the lock
    std::cout << "Preparation work\n";

    lock.lock();  // Now lock when needed
    std::cout << "Critical section\n";
    // Auto-unlocks when lock destructs
}

int main() {
    std::thread t1(deferred_lock_example);
    std::thread t2(deferred_lock_example);

    t1.join();
    t2.join();
}
```

Mutex and Locking Mechanisms - Synchronization Primitives

Code Examples

📌 7. std::lock for Deadlock-Free Multi-Mutex Locking

`std::lock()` locks all mutexes atomically without deadlock, regardless of order. `std::adopt_lock` tells guards the mutexes are already locked.

```
#include <iostream>
#include <thread>
#include <mutex>

std::mutex mtx1, mtx2;
int resource1 = 0, resource2 = 0;

void transfer(int amount) {
    std::lock(mtx1, mtx2);  // ✅ Atomic lock - no deadlock possible

    std::lock_guard<std::mutex> lock1(mtx1, std::adopt_lock);
    std::lock_guard<std::mutex> lock2(mtx2, std::adopt_lock);

    resource1 -= amount;
    resource2 += amount;

    std::cout << "Transferred " << amount << "\n";
}

int main() {
    resource1 = 1000;
    resource2 = 0;

    std::thread t1(transfer, 100);
    std::thread t2(transfer, 200);

    t1.join();
    t2.join();

    std::cout << "Resource1: " << resource1 << ", Resource2: " << resource2 << "\n";
}
```

Mutex and Locking Mechanisms - Synchronization Primitives

Code Examples

📌 8. std::scoped_lock (C++17) - Simplified Multi-Mutex

`std::scoped_lock` (C++17) combines the benefits of `std::lock()` and `std::lock_guard`, simplifying multi-mutex locking.

```
#include <iostream>
#include <thread>
#include <mutex>

std::mutex mtx1, mtx2;

void modern_multi_lock() {
    std::scoped_lock lock(mtx1, mtx2);  // ✅ C++17 - locks all, RAII, deadlock-free

    // Critical section with both mutexes held
    std::cout << "Both locks acquired\n";

    // Auto-releases both when lock goes out of scope
}

int main() {
    std::thread t1(modern_multi_lock);
    std::thread t2(modern_multi_lock);

    t1.join();
    t2.join();
}
```

What You Learned

Key Takeaways

- ✓ Mutex Overview
- ✓ The Need for Synchronization
- ✓ The Modern Way
- ✓ Mutex Types

→ Swipe to learn →